SQL SUBQUERIES — DETAILED NOTES

1. What is a Subquery?

A subquery is a SQL query written inside another SQL query.

In simple words:

> Query inside a query = Subquery

The inner query is called the subquery, and the outer query is called the main query or outer query.

Basic structure

```
SELECT column
FROM table
WHERE column = (
    SELECT column
    FROM table
    WHERE condition
);
```

Think of it like this:

```
Outer Query
    |
    |---- Subquery
              |
              |---- Produces a result
    |
    |---- Outer query uses that result
```

2. Why Do We Use Subqueries?

Suppose we have an `employees` table:

emp_idnamesalarydepartment1Rahul50000IT2Aman70000HR3Neha90000IT4Priya60000Sales

Suppose the question is:

> Find employees whose salary is greater than the average salary.

First, we need to calculate the average salary.

```
SELECT AVG(salary)
FROM employees;
```

Suppose result is:

```
67500
```

Now:

```
SELECT name, salary
FROM employees
WHERE salary > 67500;
```

But instead of manually finding `67500`, we can put the first query inside the second query.

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

This is a subquery.

3. How Does a Subquery Execute?

Consider:

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

SQL conceptually evaluates the inner query first:

```
SELECT AVG(salary)
FROM employees;
```

Result:

```
67500
```

Then the outer query becomes conceptually:

```
SELECT name, salary
FROM employees
WHERE salary > 67500;
```

Result:

namesalaryNeha90000

Important

A subquery does not always have to be executed literally first by the database engine internally. The optimizer may transform the query.

For exams, however, remember:

> Conceptually, the subquery provides the value/result that the outer query uses.

4. Types of Subqueries

Subqueries can be classified in several ways.

Based on the number of rows returned

1. Single-row subquery
2. Multiple-row subquery
3. Multiple-column subquery

Based on dependency

1. Non-correlated subquery
2. Correlated subquery

Based on location

1. Subquery in `WHERE`
2. Subquery in `FROM`
3. Subquery in `SELECT`
4. Subquery in `HAVING`

5. Single-Row Subquery

A single-row subquery returns exactly one row.

Example:

```
SELECT AVG(salary)
FROM employees;
```

Result:

```
67500
```

Because only one value is returned, we can use operators such as:

```
=
>
<
>=
<=
<>
```

Example

Find employees earning more than the average salary.

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Result

namesalaryNeha900006. Operators with Single-Row Subqueries

Suppose:

```
SELECT AVG(salary)
FROM employees;
```

returns:

```
67500
```

Then:

=

```
WHERE salary = (
    SELECT AVG(salary)
    FROM employees
)
```

Finds employees whose salary equals the average.

>

```
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
)
```

Finds employees earning more than average.

<

```
WHERE salary < (
    SELECT AVG(salary)
    FROM employees
)
```

Finds employees earning less than average.

>=

```
WHERE salary >= (
    SELECT AVG(salary)
    FROM employees
)
```

<=

```
WHERE salary <= (
    SELECT AVG(salary)
    FROM employees
)
```

7. Multiple-Row Subquery

A multiple-row subquery returns more than one row.

Example:

Suppose we have:

`departments`

dept_iddept_name1IT2HR3Sales

`employees`

namedepartmentRahulITAmanHRNehaITPriyaSales

Suppose we want employees working in departments whose names are either IT or HR.

Subquery:

```
SELECT dept_name
FROM departments
WHERE dept_id IN (1, 2);
```

Result:

```
IT
HR
```

Because multiple rows are returned, we cannot normally use:

```
=
```

Instead, use operators such as:

```
IN
ANY
ALL
EXISTS
```

8. `IN` with Subquery

`IN` checks whether a value belongs to a set of values.

Example

```
SELECT name, department
FROM employees
WHERE department IN (
    SELECT dept_name
    FROM departments
    WHERE dept_id IN (1, 2)
);
```

The subquery returns:

```
IT
HR
```

So the outer query effectively checks:

```
WHERE department IN ('IT', 'HR')
```

Result:

namedepartmentRahulITAmanHRNehaIT9.

`ANY` Operator

`ANY` means:

> The condition must be true for at least one value returned by the subquery.

Example:

Suppose salaries are:

```
50000
70000
90000
60000
```

Consider:

```
SELECT name, salary
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
```

```
    WHERE department = 'IT'
);
```

Suppose IT salaries are:

```
50000
90000
```

Then:

```
salary > ANY (50000, 90000)
```

means:

> Salary should be greater than at least one of these values.

So a salary of `60000` qualifies because:

```
60000 > 50000
```

It doesn't need to be greater than `90000`.

10. `ALL` Operator

`ALL` means:

> The condition must be true for every value returned by the subquery.

Example:

```
SELECT name, salary
FROM employees
WHERE salary > ALL (
    SELECT salary
    FROM employees
    WHERE department = 'IT'
);
```

If IT salaries are:

```
50000
90000
```

then:

```
salary > ALL (50000, 90000)
```

means:

```
salary > 50000
AND
salary > 90000
```

Therefore salary must be greater than:

```
90000
```

11. Difference Between **`ANY`** and **`ALL`**

Suppose subquery returns:

```
10
20
30
```

`> ANY`

```
x > ANY (10,20,30)
```

means:

```
x > 10
OR
x > 20
OR
x > 30
```

Effectively, for this positive `>` comparison:

```
x > 10
```

is sufficient.

`> ALL`

```
x > ALL (10,20,30)
```

means:

```
x > 10
AND
x > 20
AND
x > 30
```

Therefore:

```
x > 30
```

is required.

Easy memory trick

```
ANY = At least ONE
ALL = Every ONE
```

12. `EXISTS` Operator

`EXISTS` checks whether the subquery returns at least one row.

It does not primarily care about the actual value returned.

Example:

```
SELECT name
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM departments d
    WHERE d.dept_name = e.department
);
```

If a matching department exists, the employee is selected.

Important exam point

With `EXISTS`:

```
SELECT 1
```

is commonly used because the actual selected value is irrelevant.

The database only needs to know:

```
Does at least one matching row exist?
```

13. `NOT EXISTS`

`NOT EXISTS` means:

> Select rows for which the subquery finds no matching row.

Example:

```
SELECT name
FROM employees e
WHERE NOT EXISTS (
    SELECT 1
    FROM departments d
    WHERE d.dept_name = e.department
);
```

This finds employees whose department does not exist in the `departments` table.

14. Correlated Subquery

This is one of the most important subquery concepts for exams.

A correlated subquery is a subquery that depends on a value from the outer query.

Example:

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e1.department
);
```

Here:

```
e1.department
```

belongs to the outer query.

The inner query uses it.

Therefore the inner query is correlated with the outer query.

15. Understanding Correlated Subquery Like a Beginner

Suppose:

namedepartmentsalaryRahulIT50000NehaIT90000AmanHR70000PriyaHR60000

Question:

> Find employees earning more than the average salary of their own department.

For Rahul:

```
IT average = (50000 + 90000) / 2
           = 70000
```

Rahul:

```
50000 > 70000 □
```

For Neha:

```
90000 > 70000 □
```

For Aman:

```
HR average = (70000 + 60000) / 2
           = 65000
70000 > 65000 □
```

For Priya:

```
60000 > 65000 □
```

Result:

namesalaryNeha90000Aman70000

The inner query changes according to the current employee from the outer query.

That is why it is called correlated.

16. Non-Correlated vs Correlated Subquery

Non-correlated

```
SELECT name
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

The inner query does not depend on the outer query.

Correlated

```
SELECT e1.name
FROM employees e1
WHERE salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e1.department
);
```

The inner query depends on:

```
e1.department
```

Easy difference

FeatureNon-CorrelatedCorrelatedDepends on outer query?NoYesOuter reference inside subquery?NoYesConceptually executesIndependentlyFor each outer rowUsually easier?YesMore complex17.

Subquery in `WHERE`

This is the most common type.

Example:

> Find employees earning more than average salary.

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

18. Subquery in `FROM`

A subquery can be used as a derived table in the `FROM` clause.

Example:

```
SELECT *
FROM (
    SELECT name, salary
    FROM employees
    WHERE salary > 60000
) AS high_salary;
```

The inner query creates a temporary result set.

Conceptually:

```
employees
    ↓
Subquery
    ↓
high_salary
    ↓
Outer query
```

Important MySQL point

A derived table generally needs an alias:

```
AS high_salary
```

19. Subquery in `SELECT`

A subquery can also appear in the `SELECT` list.

Example:

```
SELECT
    name,
    salary,
    (SELECT AVG(salary) FROM employees) AS average_salary
FROM employees;
```

Result could look like:

namesalaryaverage_salaryRahul5000067500Aman7000067500Neha9000067500Priya6000067500

The subquery returns one value, which is displayed alongside every row.

20. Subquery in `HAVING`

Subqueries can also be used with `HAVING`.

Example:

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM employees
);
```

This means:

> Display departments whose average salary is greater than the overall average salary.

21. Subquery with `UPDATE`

Subqueries are not limited to `SELECT`.

Example:

> Increase salary by 10% for employees working in the IT department.

```
UPDATE employees
SET salary = salary * 1.10
WHERE department = (
    SELECT dept_name
    FROM departments
    WHERE dept_id = 1
);
```

22. Subquery with `DELETE`

Example:

> Delete employees belonging to a particular department.

```
DELETE FROM employees
WHERE department = (
    SELECT dept_name
    FROM departments
    WHERE dept_id = 3
);
```

23. Nested Subquery

A subquery can contain another subquery.

Therefore:

```
Query
 ↓
Subquery
 ↓
Sub-subquery
```

Example:

```
SELECT name
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department = (
        SELECT dept_name
```

```
        FROM departments
        WHERE dept_id = 1
    )
);
```

Here there are multiple levels.

24. Subquery vs JOIN

The same problem can often be solved using either a subquery or a `JOIN`.

Subquery

```
SELECT name
FROM employees
WHERE department IN (
    SELECT dept_name
    FROM departments
    WHERE location = 'Delhi'
);
```

JOIN

```
SELECT e.name
FROM employees e
JOIN departments d
ON e.department = d.dept_name
WHERE d.location = 'Delhi';
```

Both can produce similar results.

Important

Do not assume:

> Subquery is always slower than JOIN.

Actual performance depends on the database engine, query structure, indexes, data size, and optimizer.

25. Common Exam Trap: `=` vs `IN`

Suppose this subquery returns:

```
IT
HR
Sales
```

This is problematic:

```
WHERE department = (
    SELECT dept_name
    FROM departments
);
```

because `=` expects a single value, while the subquery returns multiple rows.

Use:

```
WHERE department IN (
    SELECT dept_name
    FROM departments
);
```

Remember

```
One value  → =

Multiple values → IN
```

26. Common Exam Trap: Aggregate Functions

Aggregate functions usually return one value.

For example:

```
SELECT MAX(salary)
FROM employees;
```

returns one value.

Therefore:

```
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

is valid.

Similarly:

```
SELECT AVG(salary)
FROM employees;
```

returns one value.

So:

```
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

is valid.

27. Find Employee with Maximum Salary

```
SELECT name, salary
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

If maximum salary is:

```
90000
```

then all employees earning `90000` are returned.

Important

This is better than assuming there is only one employee with maximum salary.

If two employees have the same maximum salary, both can be returned.

28. Find Employee with Minimum Salary

```
SELECT name, salary
FROM employees
WHERE salary = (
    SELECT MIN(salary)
    FROM employees
);
```

29. Find Employees Earning More Than Average

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Very common exam question.

30. Find Employees Earning Less Than Average

```
SELECT name, salary
FROM employees
WHERE salary < (
    SELECT AVG(salary)
    FROM employees
);
```

31. Find Second Highest Salary

One common approach:

```
SELECT MAX(salary)
```

```
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

The inner query finds the highest salary.

The outer query searches for the maximum salary below that value.

Example:

```
90000
70000
60000
50000
```

Inner:

```
MAX = 90000
```

Then:

```
MAX(salary) where salary < 90000
```

Result:

```
70000
```

32. Find Third Highest Salary

A nested approach can be used:

```
SELECT MAX(salary)
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
    WHERE salary < (
        SELECT MAX(salary)
        FROM employees
    )
);
```

Conceptually:

```
Highest
   ↓
Second Highest
   ↓
Third Highest
```

33. Find Employees Who Earn the Highest Salary in Their Department

This is a classic correlated-subquery question.

```
SELECT e1.name, e1.department, e1.salary
FROM employees e1
WHERE e1.salary = (
    SELECT MAX(e2.salary)
    FROM employees e2
    WHERE e2.department = e1.department
);
```

The inner query finds the maximum salary for the employee's department.

Then the outer query checks whether that employee has that salary.

34. Scalar Subquery

A scalar subquery returns exactly one value.

Example:

```
SELECT AVG(salary)
FROM employees;
```

This produces one value.

It can be used where a single value is expected:

```
SELECT name
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
```

```
);
```

Exam definition

> A scalar subquery returns exactly one row and one column.

35. Multi-Column Subquery

A subquery can return more than one column.

Example:

```
SELECT department, MAX(salary)
FROM employees
GROUP BY department;
```

Result:

departmentMAX(salary)IT90000HR70000

This is a multi-column result.

One way to compare multiple columns is using row constructors:

```
SELECT *
FROM employees
WHERE (department, salary) IN (
    SELECT department, MAX(salary)
    FROM employees
    GROUP BY department
);
```

This finds employees whose `(department, salary)` pair matches a department's maximum salary.

36. Important NULL Issue with `NOT IN`

This is a very important SQL exam concept.

Suppose:

```
WHERE department NOT IN (
    SELECT department
    FROM departments
);
```

If the subquery contains `NULL`, `NOT IN` can produce unexpected results because SQL uses three-valued logic:

```
TRUE
FALSE
UNKNOWN
```

Therefore, when NULLs are possible, `NOT EXISTS` is often a safer formulation.

Example:

```
SELECT e.name
FROM employees e
WHERE NOT EXISTS (
    SELECT 1
    FROM departments d
    WHERE d.dept_name = e.department
);
```

37. Subquery Execution Concept

For a simple non-correlated subquery:

```
SELECT name
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Conceptually:

```
          SUBQUERY
              ↓
        AVG(salary)
              ↓
           67500
              ↓
        OUTER QUERY
              ↓
      salary > 67500
              ↓
          final rows
```

For correlated:

```
Outer row 1
    ↓
Inner query using row 1
    ↓
comparison
    ↓
Outer row 2
    ↓
Inner query using row 2
    ↓
comparison
    ↓
...
```

38. Most Important Subquery Operators

Operator Meaning `=` Equal to one value `>` Greater than one value `<` Less than one value `>=` Greater than or equal `<=` Less than or equal `<>` Not equal `IN` Matches any value in a set `ANY` Condition true for at least one value `ALL` Condition true for every value `EXISTS` At least one row exists `NOT EXISTS` No matching row exists

39. Quick Difference: IN vs EXISTS

`IN`

Compares a value against values returned by a subquery.

```
WHERE department IN (
    SELECT dept_name
    FROM departments
);
```

`EXISTS`

Checks whether at least one matching row exists.

```
WHERE EXISTS (
    SELECT 1
    FROM departments d
    WHERE d.dept_name = e.department
);
```

Easy memory

```
IN      → "Is this value present?"
EXISTS  → "Does a matching row exist?"
```

40. Important Rules for Subqueries

Rule 1

A subquery is normally enclosed in:

```
( )
```

Rule 2

A scalar subquery should return:

```
1 row
1 column
```

when used where a single value is expected.

Rule 3

Use `IN` when multiple values may be returned.

Rule 4

Use `ANY` when the condition needs to match at least one returned value.

Rule 5

Use `ALL` when the condition must match every returned value.

Rule 6

`EXISTS` checks whether rows exist.

Rule 7

A correlated subquery references the outer query.

Rule 8

A derived table in `FROM` generally requires an alias in MySQL.

Example:

```
FROM (
    SELECT ...
) AS temp;
```

41. Exam-Oriented Questions

Q1. What is a subquery?

Answer:

A subquery is a query nested inside another SQL query. The result of the inner query is used by the outer query.

Q2. What is a correlated subquery?

Answer:

A correlated subquery is a subquery that references a column from the outer query and therefore depends on the current row of the outer query.

Q3. Which operator is commonly used when a subquery returns multiple values?

```
IN
```

Q4. Which operator checks whether a subquery returns at least one row?

```
EXISTS
```

Q5. Difference between **`ANY`** and **`ALL`**?

```
ANY → condition must be true for at least one value.

ALL → condition must be true for every value.
```

Q6. What is a scalar subquery?

A subquery that returns exactly one row and one column.

Q7. Find employees earning more than average salary.

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Q8. Find employee with maximum salary.

```
SELECT *
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

Q9. Find second-highest salary.

```
SELECT MAX(salary)
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Q10. Find employees whose salary is greater than their department average.

```
SELECT e1.name, e1.salary, e1.department
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e1.department
);
```

This is a correlated subquery.

42. One-Page Revision

Remember this structure:

```
                    SUBQUERY
                       |
                       |
          ┌────────────┴────────────┐
          ↓                         ↓
     Non-correlated             Correlated
          |                         |
 Independent              Depends on outer query
```

Based on result:

```
Single row
    ↓
=, >, <, >=, <=, <>

Multiple rows
    ↓
IN, ANY, ALL, EXISTS
```

Based on location:

```
WHERE
FROM
SELECT
HAVING
```

Most important concepts for exams:

```
1. Definition of subquery
2. Single-row subquery
3. Multiple-row subquery
4. Scalar subquery
5. IN
6. ANY
7. ALL
8. EXISTS
9. NOT EXISTS
10. Correlated subquery
11. Non-correlated subquery
12. Subquery in WHERE
13. Subquery in FROM
14. Subquery in SELECT
15. Subquery in HAVING
16. Nested subqueries
17. Subquery with UPDATE
18. Subquery with DELETE
19. Subquery vs JOIN
20. NULL + NOT IN
```

Golden memory trick

```
SUBQUERY = QUERY INSIDE QUERY

ONE VALUE  → =, >, <, >=, <=
MANY VALUES → IN, ANY, ALL
EXISTENCE → EXISTS / NOT EXISTS

CORRELATED =
INNER QUERY DEPENDS ON OUTER QUERY
```